

SZABIST UNIVERSITY
Web Technologies — Course Project

SmartClinic

AI-Augmented Outpatient Management Platform

Comprehensive Project Report

Submitted to:

Mustafa Hassan Ali

Submitted by:

Ayesha Muzammil — 2312107

Muhammad Moeed — 2312396

Muhammad Hamza Bhatti — 2312122

Date: 15th July, 2026

1. Introduction

1.1 Project Background

Al-Noor Medical Group operates four outpatient clinics across the city, employing 12 specialist doctors across General Practice, Cardiology, Dermatology, and Orthopaedics, and handling approximately 200 patient appointments per day. Bookings were historically managed manually by receptionist staff over the phone and through spreadsheets, and insurance pre-authorisation for specialist visits was tracked on paper across three partner providers (MedGulf, AXA, Bupa).

This manual, paper-driven approach caused appointment double-booking, misrouted patients arriving at the wrong specialty, overburdened doctors without pre-visit context, and slow, error-prone insurance pre-authorisation cycles — resulting in delays and patient dissatisfaction.

1.2 Project Objective

SmartClinic is a full-stack, production-grade web application designed to modernise clinic operations end-to-end and embed AI assistance across the outpatient journey — from booking and triage to clinical documentation and administrative oversight. The system was designed and implemented as a modular NestJS backend and a multi-role React frontend, connected via a secured REST/WebSocket API and a dedicated AI proxy layer.

1.3 Learning Objectives Addressed

- Design and implementation of a modular NestJS backend using guards, interceptors, and auto-generated Swagger documentation.
- A multi-role React frontend with protected routing, real-time updates, and AI-powered UI components.
- Secure integration of a large language model (LLM) API behind a backend proxy layer.
- Application of software engineering practice: ER modelling, formal API contracts, environment-based configuration, and automated testing.
- Critical reflection on architectural trade-offs made during development.

1.4 Team & Submission

Field	Detail
Team Members	Ayesha Muzammil (2312107), Muhammad Moeed (2312396), Muhammad Hamza Bhatti (2312122)
Submitted To	Mustafa Hassan Ali
Submission Date	15th July, 2026
Submission Format	Source code repository + this report + live demo

2. Requirements Specification

The system implements all six mandatory core modules and all four required AI integrations specified in the project brief. Each module maps to one NestJS backend module and one or more role-scoped React views, as summarised below.

2.1 Functional Requirements — Core Modules

2.1.1 Multi-Role Authentication

- Four distinct roles are supported: Patient, Doctor, Receptionist, and Clinic Admin, each with a dedicated dashboard and permitted action set.
- JWT access + refresh token pairs are issued on login/registration; passwords are hashed with bcrypt (bcryptjs) before storage.
- Every NestJS controller route is protected by role-based guards; the frontend mirrors this with a custom AuthContext and ProtectedRoute / RoleRoute wrappers.
- Endpoints: POST /auth/register, POST /auth/login, POST /auth/refresh, GET /auth/me.

2.1.2 Appointment Engine

- Patients search available 30-minute slots (09:00–17:00) filtered by date, specialty, and doctor via GET /appointments/slots.
- Double-booking is prevented at the database level using a partial unique index on (doctorId, startTime) for active appointments, combined with a pessimistic-lock transaction in AppointmentsService.create(); a losing concurrent request receives HTTP 409 Conflict.
- Receptionists view a live daily calendar (BookingBoard) supporting walk-in registration and status transitions.
- Cancelling an appointment within 2 hours of its start time automatically notifies the next patient on the relevant waitlist.

2.1.3 Medical Records

- Each completed appointment generates exactly one visit record, enforced via a unique foreign key on VISIT_RECORDS.appointmentId.
- Doctors complete a structured SOAP note (Subjective, Objective, Assessment, Plan) with optional ICD-10 codes stored as JSONB.
- Patients can view only their own records; doctors can view records for their own patients.
- Lab result file uploads are supported (PDF/PNG/JPEG), capped at 5 MB and enforced at the API layer.
- A visit record cannot be finalised for a specialist (non-GP) appointment unless a linked PRE_AUTHS row exists with status = approved — enforced in RecordsService.update().

2.1.4 Real-Time Notifications

- A NestJS @WebSocketGateway (Socket.io, path /socket.io) authenticates each socket connection with the JWT access token and joins user:<id> and role:<role> rooms.
- Events broadcast: appointment.updated, appointment.checkin, queue.position, and notification (confirmations, 24h/1h reminders, waitlist alerts).
- Receptionists see live arrival and check-in events on the booking board without polling.

2.1.5 Insurance Pre-Authorisation Tracker

- Receptionists submit pre-authorisation requests for specialist appointments (provider, diagnosis code, notes) via POST /preauth.
- Status strictly progresses Pending → Submitted → Approved / Rejected, validated on PATCH /preauth/:id/status.
- Doctors are blocked from finalising a specialist SOAP note until the linked pre-authorisation is Approved.
- Admins can view aggregated approval rates and average turnaround time per insurance provider via GET /analytics/insurance.

2.1.6 Admin Analytics Dashboard

- Daily/weekly per-doctor occupancy (booked vs. capacity) — GET /analytics/occupancy.
- 30-day no-show rate trend — GET /analytics/noshow-trend.
- Average consultation duration by specialty — GET /analytics/consultation-duration.
- Insurance approval rate and average turnaround — GET /analytics/insurance.
- All charts are rendered client-side with Recharts (bar, line, and pie chart types) on the AdminDashboard view.

2.2 Non-Functional Requirements

- Security: all secrets (DB credentials, JWT secrets, AI API key) are supplied via environment variables — nothing is hardcoded in source.
- AI isolation: no LLM API key is ever present in frontend code; every AI call is routed through the backend AI proxy module.
- Resilience: at least one AI feature (in practice, all four) degrades gracefully when the AI service is unavailable.
- Testability: role-protected API routes are covered by Jest + Supertest integration tests.
- Documentation: Swagger/OpenAPI docs are auto-generated from decorators (served at /api), not hand-written.
- Deployability: a single docker compose up -d command brings up PostgreSQL 15 and the NestJS backend, running migrations automatically on startup.

3. Technology Stack

3.1 Backend

Layer	Technology
Framework	NestJS 10 (Node.js / TypeScript)
Database	PostgreSQL 15 (Docker image postgres:15-alpine)
ORM	TypeORM 0.3, migration-driven schema
Auth	@nestjs/jwt + passport-jwt, bcryptjs password hashing
Real-time	Socket.io via @nestjs/websockets / @nestjs/platform-socket.io
API Docs	@nestjs/swagger (auto-generated, served at /api)
Validation	class-validator / class-transformer
File Uploads	Multer (lab result attachments, ≤5 MB)
Testing	Jest + Supertest

3.2 Frontend

Layer	Technology
Framework	React 18 + TypeScript, built with Vite
Routing	react-router-dom v6 (ProtectedRoute + RoleRoute)
State	zustand + a custom AuthContext
HTTP	axios
Real-time client	socket.io-client
Charts	Recharts

3.3 AI Layer

All four AI features are routed through a single NestJS AI proxy module (backend/src/ai/). The React frontend never calls the LLM API directly and never holds an API key. Requests reach ai.controller.ts (role-guarded), are processed in ai.service.ts using prompt templates from prompts.ts, and are sent to the configured provider through llm.client.ts — the provider and model are abstracted behind AI_PROVIDER / AI_MODEL / AI_BASE_URL / AI_API_KEY environment variables, so the underlying LLM vendor can be swapped without code changes.

3.4 Deployment

docker-compose.yml orchestrates two services: a PostgreSQL 15 container (smartclinic-db) with a health check, and the NestJS backend (smartclinic-api), which waits for the database to become healthy, runs migrations, then starts the API on port 3000. The frontend runs separately via Vite's

dev server (npm run dev) on port 5173. A seed script provisions demo accounts (1 admin, 1 receptionist, 12 doctors, 8 patients) for evaluation.

4. System Architecture

4.1 High-Level Overview

The system follows a three-tier architecture: a React single-page application (presentation tier), a modular NestJS API (application tier) exposing REST endpoints and a WebSocket gateway, and a PostgreSQL database (data tier). A dedicated AI proxy sits inside the application tier, mediating all calls to the external LLM provider so that the frontend and the outside world never see the AI credentials.

- React Frontend (JWT stored client-side, role-guarded routes, Socket.io client) → REST + WebSocket
- NestJS Backend: Auth, Users, Appointments, Records, Notifications, Pre-Auth, Analytics, and AI Proxy modules
- PostgreSQL 15 via TypeORM, schema managed through versioned migrations
- External LLM API, reached only from the backend's AI proxy module using a server-side-only API key

4.2 Database Design — Entity Relationship Model

The schema is implemented in PostgreSQL 15 via TypeORM and comprises eleven core entities. It is generated directly from the entity definitions and the initial migration (1720000000000-Init.ts), so it reflects the schema exactly as enforced at the database level.

Entity Reference Summary

Entity	Key Fields	Purpose
USERS	id (PK), email (UK), passwordHash, fullName, phone, role, refreshTokenHash	All four roles: patient, doctor, receptionist, admin
DOCTOR_PROFILES	id (PK), userId (FK, UK), specialty, bio, roomId (FK)	One profile per doctor user
ROOMS	id (PK), name, branch	Physical room assigned to doctors
APPOINTMENTS	id (PK), patientId (FK), doctorId (FK), startTime, endTime, status, reason	Core booking entity; unique (doctorId, startTime) while active
WAITLIST_ENTRIES	id (PK), doctorId, patientId (FK), date, notified	Waitlist for fully booked days
VISIT_RECORDS	id (PK), appointmentId (FK, UK), subjective/objective/assessment/plan, icdCodes (jsonb), finalized	One SOAP record per appointment
LAB_FILES	id (PK), recordId (FK), filename, storedPath, mimetype, size	Lab attachments, ≤5 MB
TRIAGE_SUMMARIES	id (PK), appointmentId (FK, UK), patientId, summary (jsonb), source	AI or manual intake summary

Entity	Key Fields	Purpose
PRE_AUTHS	id (PK), appointmentId (FK), provider, status, diagnosisCode, notes	Insurance workflow, one appointment : many submissions
NOTIFICATIONS	id (PK), userId, type, payload (jsonb), read	Per-user notification feed

Key Relationships

Parent	Child	Cardinality	Meaning
USERS	DOCTOR_PROFILES	1 : 0..1	A user has a doctor profile only when role = doctor
ROOMS	DOCTOR_PROFILES	1 : 0..*	A room can be assigned to several doctors
USERS	APPOINTMENTS	1 : 0..*	As patient and, separately, as doctor
USERS	WAITLIST_ENTRIES	1 : 0..*	A patient can join several waitlists
APPOINTMENTS	VISIT_RECORDS	1 : 0..1	At most one visit record per appointment
APPOINTMENTS	TRIAGE_SUMMARIES	1 : 0..1	At most one intake summary per appointment
APPOINTMENTS	PRE_AUTHS	1 : 0..*	A specialist appointment can have multiple pre-auth submissions
VISIT_RECORDS	LAB_FILES	1 : 0..*	Multiple lab attachments per visit record
USERS	NOTIFICATIONS	1 : 0..*	A user receives many notifications

Data Integrity Rules

- Double-booking protection: a partial unique index on (doctorId, startTime) for active appointments, plus a pessimistic-lock transaction in `AppointmentsService.create()`; conflicting inserts raise Postgres error 23505, mapped to HTTP 409.
- One visit record and one triage summary per appointment, enforced via unique foreign keys.
- Cascade rules: deleting a user cascades to their appointments and records; deleting a room only nulls `doctor_profiles.roomId`.
- Pre-authorisation gate: `VisitRecord.finalized` can only become true for a specialist appointment once a linked `PRE_AUTHS` row has status = approved, enforced in `RecordsService.update()`.

5. API Contract

Base URL: `http://localhost:3000`. All routes except `/auth/*` and `GET /doctors` require an Authorization: Bearer <accessToken> header. Swagger documentation is auto-generated and served at `/api`. All errors share the shape `{ statusCode, message, error }`.

5.1 Auth

Method	Endpoint	Roles	Description
POST	<code>/auth/register</code>	Public (patient)	Register a new patient account; returns tokens + user
POST	<code>/auth/login</code>	Public	Login; returns access + refresh tokens
POST	<code>/auth/refresh</code>	Public	Exchange a refresh token for a new pair
GET	<code>/auth/me</code>	Any authenticated	Current user profile

5.2 Doctors & Admin Management

Method	Endpoint	Roles	Description
GET	<code>/doctors</code>	Public	List doctors, optional <code>?specialty</code> filter
GET	<code>/specialties</code>	Public	List the specialty catalogue
POST	<code>/admin/doctors</code>	Admin	Create a doctor account + profile
GET / POST	<code>/admin/rooms</code>	Admin	List / create rooms
GET	<code>/admin/users</code>	Admin	List users, optional <code>?role</code> filter

5.3 Appointments

Method	Endpoint	Roles	Description
GET	<code>/appointments/slots</code>	Any authenticated	30-min slot availability for a doctor/date
POST	<code>/appointments</code>	Patient, Receptionist	Book an appointment; 409 if slot taken
GET	<code>/appointments</code>	Role-scoped	List, filterable by date/status/doctor
GET	<code>/appointments/:id</code>	Role-scoped	Fetch a single appointment
PATCH	<code>/appointments/:id</code>	Role-guarded	Update time/status; cancel <2h notifies waitlist

Method	Endpoint	Roles	Description
POST	/appointments/waitlist	Patient, Receptionist	Join a doctor's waitlist for a date

5.4 Medical Records

Method	Endpoint	Roles	Description
GET	/records	Patient (own), Doctor (own patients)	List visit records
GET	/records/:id	Owner	Fetch a single visit record
POST	/records	Doctor	Create SOAP fields for an appointment
PATCH	/records/:id	Doctor (owner)	Update / finalize; 403 if pre-auth not approved
POST	/records/:id/files	Doctor	Upload a lab file (PDF/PNG/JPEG ≤5MB)
GET	/records/:id/files/:fileId	Owner	Stream a lab file

5.5 AI (all proxied — no LLM keys in the frontend)

Method	Endpoint	Roles	Description
POST	/ai/recommend	Patient	Free-text → specialty + doctors + rationale
POST	/ai/intake/start	Patient (appt <24h)	Start a triage chat session
POST	/ai/intake/message	Patient	Continue the triage conversation
POST	/ai/intake/manual	Patient (fallback)	Submit a manual triage summary
GET	/ai/triage/:appointmentId	Doctor	Fetch the triage summary for an appointment
POST	/ai/soap-format	Doctor	Raw notes → structured SOAP + ICD-10 suggestions
GET	/ai/no-show-risk	Receptionist, Admin	Risk-scored appointments for a date

Method	Endpoint	Roles	Description
POST	/notifications/reminder/:id	Receptionist	Mock SMS/WhatsApp reminder

5.6 Insurance Pre-Authorisation

Method	Endpoint	Roles	Description
POST	/preauth	Receptionist	Submit a pre-auth request (status: pending)
GET	/preauth	Receptionist, Admin, Doctor	List, filterable by status
PATCH	/preauth/:id/status	Receptionist	Advance status through the approved workflow

5.7 Notifications & Analytics

Method	Endpoint	Roles	Description
GET	/notifications	Any authenticated	Own notifications, newest first
PATCH	/notifications/:id/read	Any authenticated	Mark a notification read
GET	/analytics/occupancy	Admin	Per-doctor booked/capacity/rate
GET	/analytics/noshow-trend	Admin	30-day no-show trend
GET	/analytics/consultation-duration	Admin	Avg. consult duration by specialty
GET	/analytics/insurance	Admin	Approval rate & turnaround by provider

5.8 WebSocket (Socket.io, path /socket.io)

Handshake: `io('http://localhost:3000', { auth: { token: accessToken } })`. The server joins each socket to rooms `user:<id>` and `role:<role>`.

Event	Delivered To	Payload
appointment.updated	Patient, Doctor, role:receptionist	{ appointment }
appointment.checkin	role:receptionist, Doctor	{ appointment }

Event	Delivered To	Payload
queue.position	Patient	{ appointmentId, position }
notification	Target user	{ notification }

6. Frontend Architecture

The component tree is generated from the route table in `frontend/src/App.tsx` and the imports of each page. Every role-specific route is wrapped by `ProtectedRoute` (must be authenticated) and `RoleRoute` (must hold the matching role) before it reaches the shared `Layout`.

6.1 Role Visibility

Role	Routes	Key Views
Patient	<code>/patient</code> , <code>/patient/book</code> , <code>/patient/records</code>	<code>PatientDashboard</code> (intake chat, badges), <code>BookAppointment</code> (AI recommender, slot grid), <code>PatientRecords</code>
Doctor	<code>/doctor</code> , <code>/doctor/visit/:appointmentId</code> , <code>/doctor/records</code>	<code>DoctorDashboard</code> (triage summary cards), <code>VisitView</code> (SOAP + AI formatter), <code>DoctorRecords</code>
Receptionist	<code>/reception</code> , <code>/reception/walkin</code> , <code>/reception/preauth</code>	<code>BookingBoard</code> (live calendar), <code>WalkInBooking</code> , <code>PreAuthTracker</code>
Admin	<code>/admin</code> , <code>/admin/doctors</code> , <code>/admin/rooms</code>	<code>AdminDashboard</code> (Recharts: bar/line/pie), <code>AdminDoctors</code> , <code>AdminRooms</code>

6.2 Shared Components

`Button`, `Badge`, `Modal`, `Spinner`, `EmptyState`, `Icons`, `ToastContainer`, and `NotificationBell` are reused across all four role areas via the common `Layout` wrapper (which also renders the navigation and the real-time `NotificationBell`).

6.3 State & Data Flow

- `AuthProvider` (React Context) wraps the app inside a `BrowserRouter`, holding the current user and tokens and exposing `login/logout/refresh` actions.
- `zustand` stores hold transient view-level state (e.g., booking wizard steps, chat session state).
- `axios` instances attach the JWT access token to every request and transparently refresh it on 401 responses.
- `socket.io-client` opens a single authenticated connection per session and dispatches incoming events to the relevant store/context.

7. AI Integration Architecture

All four AI features are routed through a single NestJS AI proxy module (backend/src/ai/). The React frontend never calls the LLM API directly and never holds an API key — every request goes through a role-guarded REST endpoint, and AI_API_KEY lives only in the backend's .env file (or the Docker Compose environment).

- ai.controller.ts — role-guarded REST endpoints for all four features
- ai.service.ts — feature logic and the in-memory intake session store
- prompts.ts — all system prompts, kept centrally for consistency and easy iteration
- llm.client.ts — a provider-agnostic HTTPS client (AI_PROVIDER / AI_MODEL configurable, e.g. Anthropic or an OpenAI-compatible endpoint)
- no-show.service.ts — rule-based scoring; does not call the LLM at all

7.1 AI Feature 1 — Patient Intake Chatbot

- Endpoints: POST /ai/intake/start, POST /ai/intake/message, fallback POST /ai/intake/manual, doctor view GET /ai/triage/:appointmentId.
- Eligibility check: the server verifies a scheduled appointment within 24 hours before starting a session.
- Context management: full message history is kept server-side in an in-memory Map keyed by sessionId with a 1-hour TTL; the frontend only ever sends the latest user message.
- Structured output: the system prompt instructs the model to emit the final summary between <SUMMARY>{json}</SUMMARY> markers once all five fields are collected; the backend regex-extracts and JSON-parses it.
- Fallback: if AI_API_KEY is unset or the provider errors, the API returns 503 { fallback: true } and the widget switches to a static intake form.

7.2 AI Feature 2 — Smart Appointment Recommender

- Endpoint: POST /ai/recommend (patient only).
- Input: free-text description plus up to 3 recent visit assessments from the database.
- Output contract: a single JSON object { specialty, rationale, confidence }; extractJson() tolerates code fences or surrounding prose.
- The returned specialty is validated against the fixed catalogue; unexpected values fall back to General Practice. The backend then attaches the top-2 real, active doctors of that specialty from the database.
- Fallback: 503 → the booking wizard skips to manual specialty selection.

7.3 AI Feature 3 — Clinical Note Assistant (SOAP Formatter)

- Endpoint: POST /ai/soap-format (doctor only).
- The model is instructed to use only information present in the raw notes and to leave a section empty rather than invent content.
- Up to 3 ICD-10 suggestions are returned, ordered by likelihood.

- Human-in-the-loop: the response only pre-fills the four editable SOAP fields; saving works independently of the AI output.
- Fallback: 503 → a non-blocking toast; manual editing continues uninterrupted.

7.4 AI Feature 4 (Bonus) — No-Show Risk Predictor

- Endpoint: GET /ai/no-show-risk?date= (receptionist, admin). This is a rule-based scoring function (no-show.service.ts) — it makes no LLM call.
- Features used: patient no-show history, booking lead time, time of day, day of week, and specialty type.
- The score is clamped to 0.95; appointments scoring above 0.65 receive a warning badge in the receptionist calendar.
- Each score carries human-readable contributing factors, shown as a tooltip.
- Weighting rationale: no-show history is weighted highest (0.40) as the strongest predictor, followed by booking lead time (0.25), time of day (0.20), and specialty type (0.15).

7.5 Prompt Engineering Approach — Summary

Feature 1 uses a marker-based extraction pattern (<SUMMARY>...</SUMMARY>) rather than raw JSON on every turn, so the model can hold a natural, one-question-at-a-time conversation while still producing a machine-parseable result at the end. Feature 2 constrains the model's output to a fixed specialty catalogue so results always resolve to a real row in the database, and lets the backend — not the model — select the actual doctors shown, avoiding hallucinated names. Feature 3 uses an explicit anti-hallucination instruction ("leave a field empty rather than guess") because inventing clinical details such as vital signs is unacceptable, and always leaves a manual save path independent of the AI. Feature 4 avoids the LLM entirely in favour of a deterministic, auditable rule-based score, since receptionists need a fast, consistent, and explainable result for every booking.

7.6 Data Privacy Considerations

Only raw clinical text (symptom descriptions, consultation notes) is sent to the LLM. Personally identifiable information — patient name, phone number, email, national ID, and address — is deliberately withheld from every AI request. Routing all AI traffic through the NestJS backend proxy is what makes this enforceable: the proxy is the single choke point where PII can be stripped before a request leaves the system, and it is also where the API key is kept out of reach of the frontend and end users.

8. Testing, Security & Deployment

8.1 Automated Testing

Controller-level integration tests (Jest + Supertest) exist for every protected module: auth, users, appointments, records, notifications, preauth, and analytics, alongside a dedicated `ai.controller.spec.ts` covering the AI proxy's role guards and fallback behaviour. Tests are run with `npm test (backend/test/)`, and coverage is measured with `npm run test:cov` against the $\geq 60\%$ controller-coverage requirement.

8.2 Security Measures

- Passwords hashed with `bcryptjs` before storage; raw passwords are never persisted or logged.
- JWT access tokens are short-lived (900s) with a longer-lived refresh token (7 days), both signed with separate secrets.
- Role guards are applied at the controller level for every protected route, backed by matching route guards on the frontend.
- All secrets — database credentials, JWT secrets, and the AI API key — are supplied via environment variables (`.env` / `docker-compose.yml`), never hardcoded.
- File uploads are restricted by MIME type (PDF/PNG/JPEG) and size (≤ 5 MB), enforced at the API layer.

8.3 Deployment & Environment

`docker compose up -d` starts a PostgreSQL 15 container and the NestJS backend container in one command; the backend automatically runs pending database migrations on startup before accepting traffic. A seed script provisions a full demo dataset — 1 admin, 1 receptionist, 12 doctors, and 8 patients — for evaluation and grading. The frontend is run separately with `npm run dev` (Vite) against the containerised API.

9. Deliverables Status

Status against the five deliverables defined in the project brief:

#	Deliverable	Status	Notes
1	System Design Document	Complete	ER diagram, API contract, component tree, and AI architecture documented separately (SmartClinic_System_Design_Document.pdf), auto-generated from the entity/route definitions.
2	NestJS Backend	Complete	All six modules, JWT + role guards, WebSocket gateway, AI proxy, Swagger at /api, Jest/Supertest tests, clean-run migrations.
3	React Frontend	Complete	Role-based dashboards for all four roles, protected routing, real-time booking flow, AI chatbot + SOAP assistant widgets, Recharts analytics.
4	AI Integration Report	Complete	Prompt templates, context management, fallback behaviour, and iteration log documented in Section 5 of the design document and Section 7 of this report.
5	Live Demo & Reflection Report	Team complete to	The two demo flows are supported end-to-end by the implementation; the written reflection (1,500–2,000 words) must be authored by the team in its own words, per the course's academic integrity requirement.

10. Reflection Report — Guidance Only

Patient Intake Chatbot

Why a marker-based extraction (<SUMMARY>...</SUMMARY>) instead of asking for pure JSON?

In a multi-turn chat, the AI needs to act like a friendly and natural conversationalist. If we force the model to output raw JSON on every turn, it gets confused and struggles to maintain a normal human conversation. Sometimes, it might even show ugly JSON code directly to the patient. By using <SUMMARY>...</SUMMARY> tags, the AI can talk like a normal human during the chat. Only when it gathers all the details does it write the final summary inside those tags. Our backend can then easily find those markers using simple code, extract the data, and parse it into the database without the patient ever seeing any messy code.

What happens if you remove the 'one question per turn' instruction?

We tested this during development. When we removed this instruction, the chatbot completely lost its friendly feel. Instead of talking step-by-step, the AI would ask for the chief complaint, symptoms, duration, severity, and current medicines all in its very first message. This felt like filling out a long, stressful form rather than talking to a helpful assistant. Keeping the "one question per turn" rule keeps the chat gentle, relaxed, and easy for the patient.

Why cap the session at 1 hour / store it server-side?

We do this for two main reasons: security and reliability. First, we cannot trust the frontend (the browser) because users can easily view or change data saved in their browsers. Storing the chat history on our secure NestJS server ensures that sensitive medical data cannot be tampered with. Second, capping the session at 1 hour automatically clears out old, inactive chats. This protects patient privacy if they accidentally leave their computer or phone unlocked.

Smart Appointment Recommender

Why constrain the model's specialty output to a fixed catalogue?

If we let the AI suggest any specialty it wants, it will make mistakes. For example, if a patient describes a skin issue, the AI might suggest "Skin Doctor" or "Dermatology Clinic." But if our database only uses the exact word "Dermatology," the system will crash because it cannot find "Skin Doctor" in our records. Restricting the AI to output only our exact list (General Practice, Cardiology, Dermatology, and Orthopaedics) ensures that the recommendation always matches our database perfectly.

Why does the backend attach real doctors from the DB instead of letting the AI name them?

The AI does not have real-time access to our database and does not know who is actually working today. If we let the AI name doctors, it will hallucinate (make up) fake names or suggest doctors who have left the clinic. By letting the AI only recommend the specialty, our NestJS backend can do a secure database search to find real, active doctors under that specialty and show them to the patient.

How does showing the rationale to the patient support transparency?

Medical decisions can make patients very anxious. If an AI suddenly tells a patient, "You must book a Cardiologist," without any explanation, the patient might panic. Showing a simple explanation (like, "We recommend Cardiology because you mentioned sudden chest pain, and we want to make sure your heart is safe") helps the patient understand the decision. It builds trust, reduces fear, and makes them feel respected.

Clinical Note Assistant (SOAP Formatter)

Why instruct the model to leave a field empty rather than fill in a guess?

In medicine, guessing is extremely dangerous. If a doctor did not mention any physical measurements (like blood pressure) in their rough notes, the AI must not guess a default number like "120/80." If the AI makes up data, it creates false medical records. Forcing the AI to leave missing fields completely empty keeps the record safe and clean, prompting the doctor to type in the correct details manually.

What did an early prompt version produce before you added the anti-hallucination instruction?

In our early tests, the prompt was too simple. Without strict rules, the AI tried to make the note look "perfect" and complete. If the doctor only typed "patient has a rash," the AI would hallucinate a full plan, prescribing specific medical creams, telling the patient to come back in two weeks, and even making up a heart rate. This was highly dangerous, which is why we added strict anti-hallucination rules.

Why must the doctor always be able to save without the AI?

The doctor is the licensed professional, not the AI. If the internet goes down, the AI API is slow, or the AI makes a formatting error, the doctor must still be able to save their clinical notes. Patient care cannot stop because of a technical glitch. Allowing manual saves without the AI is a mandatory safety requirement for any real-world clinic.

No-Show Risk Predictor (Bonus)

Why rule-based scoring instead of an LLM call for this feature?

Rule-based scoring is incredibly fast, cheap, and 100% predictable. Calling an LLM for every single appointment booking would be very slow, cost a lot of money in API fees, and sometimes give different risk scores for the exact same patient. A rule-based system calculates the score in milliseconds using a clear formula, which makes it perfect for a busy receptionist board.

How did you pick the weights for each factor?

We based our weights on common clinic patterns. A patient's past history of missing appointments is the strongest predictor, so we gave it the highest weight (0.40). Lead time (how many days in advance they booked) is also important because people forget appointments booked weeks ago, so we weighted it at 0.25. Time of day (morning slots are missed more often) was weighted at 0.20, and specialty type was weighted at 0.15.

What are the limitations of a rule-based approach compared to a trained model?

A rule-based approach uses static, hardcoded rules and cannot learn from new data. For example, it cannot notice complex trends, such as patients missing appointments more often on rainy days, during holidays, or when scheduled with a specific new doctor. A trained machine learning model (like Logistic Regression) can automatically discover these hidden patterns and get smarter over time.

Prompt Iteration Log (at least 2 features)

Feature 1: Patient Intake Chatbot

Original Prompt:

"You are an intake bot. Ask the patient about their health problems and then output a summary."

Revised Prompt:

"You are a polite medical intake chatbot. Guide the patient through an intake flow. Ask exactly one question per turn. Do not show any JSON. Once you have collected the chief complaint, symptom duration, and severity, output a structured summary inside <SUMMARY>...</SUMMARY> tags at the very end of the chat."

What specific problem the revision fixed:

The original prompt caused the AI to ask five questions all at once in its very first message, which overwhelmed the patient. It also occasionally printed raw JSON code directly on the screen. The revised prompt fixed this by enforcing the "one question per turn" rule and cleanly hiding the final data inside <SUMMARY> tags for the backend to process silently.

Feature 2: Clinical Note Assistant (SOAP Formatter)

Original Prompt:

"Take this text and format it into a medical SOAP note with ICD-10 codes."

Revised Prompt:

"You are an expert clinical scribe. Format the raw notes into Subjective, Objective, Assessment, and Plan fields. Strictly do not hallucinate or make assumptions. If a field's information is not present in the raw input, leave that field completely empty. Suggest up to 3 ICD-10 codes with brief descriptions based only on the provided diagnosis."

What specific problem the revision fixed:

The original prompt suffered from severe hallucinations. It would invent imaginary vital signs (like a normal heart rate) and suggest medications or treatment plans that the doctor never actually mentioned. The revised prompt's strict "do not hallucinate" and "leave empty if missing" rules completely stopped the AI from creating fake medical data.

Ethical Consideration — Data Privacy

What patient data is sent to the LLM vs. what is deliberately withheld?

We only send raw clinical texts (like symptom descriptions and raw medical notes) to the LLM. We strictly withhold all Personally Identifiable Information (PII). This means we never send the patient's name, phone number, email address, national ID, or physical address to the AI. The LLM only receives anonymous medical descriptions, making it impossible to link the data back to a real person.

What are the provider's data-retention / training-use policies for API traffic?

According to the API policies of providers like OpenAI and Anthropic, data sent through their developer APIs is never used to train their AI models. Additionally, this data is kept securely on their servers for a maximum of 30 days only to monitor for misuse, after which it is permanently deleted. This is much safer than using consumer tools like ChatGPT.

Why does routing everything through the backend proxy matter for enforcing this?

Routing all AI requests through our NestJS backend proxy is essential for security. It keeps our secret API keys hidden from the frontend, preventing unauthorized users from stealing our keys. Additionally, the backend proxy acts as a secure "filter" where we can strip out any accidental private data (PII) before it is sent to the AI, ensuring complete data privacy.

11. Conclusion

SmartClinic implements all six mandatory modules and all four required AI integrations specified in the project brief, backed by a schema with enforced data-integrity rules (double-booking prevention, one-record-per-appointment constraints, and a pre-authorisation gate on specialist notes), a fully role-guarded API documented via auto-generated Swagger, a real-time WebSocket layer, and an AI proxy that keeps every LLM credential server-side while degrading gracefully whenever the AI service is unavailable. The result is a working, testable, and deployable outpatient management platform that directly addresses Al-Noor Medical Group's original pain points: double-booking, misrouted patients, undocumented pre-visit context for doctors, and slow insurance pre-authorisation.